



Natural Language Processing

n-gram language models

Oct 2025

Maria Schoinaki

Electra Papadopoulou

Leonidas Kontogiannis

You can view the full code implementation in the following link:



For this assignment we used the “*Brown*” corpus from the *NLTK* library. From the available categories, we selected the “*science_fiction*” subset, as it provides a coherent domain of narrative text with consistent vocabulary and syntax. This subset was used as the basis for training (80%), development (10%), and testing (10%) of both the bigram and trigram language models.

Contribution of each member

All members have contributed equally in all tasks, markdown and report writing, synchronously throughout the assignment. However, if we are to be more specific, each group member was mainly responsible for the following:

- **Leonidas Kontogiannis:** Data Preprocessing(tokenization, word/n-gram counting), Model Training, Probability Estimation(a-add smoothing), subquestion (i)
 - **Electra Papadopoulou:** Cross Entropy, Perplexity Calculation, subquestion (ii)
 - **Maria Schoinaki:** A-fine tuning, Autocompletion incomplete sentence,greedy decoding, beam search decoding, subquestion (iii)
-

Data Preprocessing

The implemented class `text_preprocessor` helps us preprocess the given corpus with:

- `corpus_tokenizer` method: responsible for splitting the corpus into sentences, and the produced sentences into words (tokens).
- `create_vocabulary` method: responsible for creating the vocabulary. The vocabulary is constructed from the training set by retaining only those words that occurred at least a specified minimum number of times. In this experiment, the

occurrence threshold was set to 10. This filtering process ensures that the vocabulary emphasizes more on frequently used words, thereby improving the robustness and generalization of the language model.

- `replace_with_unk` method: responsible for replacing all words that are not present in the constructed vocabulary with the special token ***UNK*** (unknown word). This step is crucial for handling **out-of-vocabulary (OOV)** words that may appear in the development or test datasets but were not seen frequently enough in the training data.

By introducing the ***UNK*** token, the model can assign a non-zero probability to unseen words during evaluation, preventing issues such as zero-probability sequences and infinite perplexity. This also improves the model's ability to generalize to new text containing rare or previously unseen words.

Experimental Setup

- **Corpus:** NLTK Brown, `'science_fiction'` category only.
 - **Tokenizer:** `'nltk.tokenize.TweetTokenizer'` (preserves case and punctuation tokens).
 - **Splits:** *80% train, 10% dev, 10% test* (`random_state = 40`).
 - **Padding:** Each sentence padded as
 bigram: `'[ *start*, x1, ..., xm, *end* ]'`
 trigram: `'[ *start1*, *start2*, x1, ..., xm, *end* ]'`.
 - **Vocabulary:** Words with frequency ≥ 10 in the training set, plus `{*end*, *UNK*, *start*, *start1*, *start2*}`.
 - The **same vocabulary** is used by both bigram and trigram models; $|V|$ excludes start tokens.
-

The Language Model Class

To represent our ngram Language Model, we have created the class `NgramLanguageModel`. It contains a set of methods that in total represent the inner workings of a model for sentences. More specifically:

- `train_ngram` method: responsible for initializing the model with the appropriate vocabulary and frequency counts derived from the training corpus. It has the following inputs:
 - A set of vocabulary items named `vocabulary`.
 - A ngram `Counter`
 - A (n-1)gram `Counter`

Then it is ensured that all the special tokens are included in the vocabulary. The vocabulary size $|V|$ is computed by excluding all start tokens while including `*end*` and `*UNK*`. The method then stores the ngram and context (n-1)gram frequency distributions for later use in probability estimation.

- `prob` method: returns the **conditional probability** of a given token given its context, denoted as $P(w_i|h)$ where h represents the preceding (n-1)-gram context. This probability is computed using **add- α (Laplace) smoothing**. Before computation, the method normalizes tokens by mapping any OOV word to `*UNK*`. This guarantees that all words, even those unseen in the training data, receive a valid probability estimate.
- `log_prob` method: returns the **logarithm of the conditional probability** of a token given its context. It's important to use log probabilities to prevent numerical underflow in long sequences, and it is more helpful for beam search, cross-entropy and perplexity definitions later on.
- `sentence_log_prob` method: computes the log probability of an entire sentence. This is done by summing all the log probabilities of all individual words within it. First, the appropriate start and end tokens are added, however, during iteration, the method skips `*start*` tokens since these are not predicted by the model, but it includes `*end*` tokens to reflect sentence termination probabilities.
- `fine_tune` method: optimizes the smoothing parameter α , based on how it performs on a development corpus. We test between the range 0.001 - 1.0, and select the value that minimizes the perplexity of the corpus. This is necessary for enhancing the language model's performance.

Additionally, for our own case, we utilize the `_map_oov` method that maps unseen words to `*UNK*`.

Autocompletion

We implemented 2 methods for the autocompletion task.

1. Greedy decoding (`greedy_autocomplete`)

Autocompletes a given sequence of words (prefix) using an n-gram language model, generating 1 word at a time. At each step the model:

- Builds the context of the last (n-1) tokens (padding with start tokens if needed)
- Scores all possible next tokens using the conditional probability.
- Selects the single most probable token (excluding **UNK** and start tokens).

Generation stops when the **end** token appears or the maximum sequence length is reached.

2. Beam Search (`beam_search_autocomplete`)

Autocompletes a sentence using beam search with n-gram scoring.

- It expands each partial sequence by considering the top $\text{beam_width} \times 2$ candidate (`get_next_word_candidates`) continuations and keeps only the beam_width best sequences according to their cumulative *$\log_2$ probability*.

This method typically produces more coherent and diverse completions than greedy decoding.

Building the Language Models

We implement add- α (Laplace) smoothed bigram (n=2) and trigram (n=3) language models in `NgramLanguageModel`. Both models share the same vocabulary built from the training set (≥ 10 frequency), augmented with $\{\text{*start*}, \text{*start1*}, \text{*start2*}, \text{*end*}, \text{*UNK*}\}$ (start tokens excluded from the predictable set $|V|$).

We compute frequency counts with padding:

- **Bigrams:** counts of (w_1, w_2) and contexts (w_1) .
- **Trigrams:** counts of (w_1, w_2, w_3) and contexts (w_1, w_2) .

Probability estimation (a-add Laplace Smoothing)

For context **h** (*length* $n-1$) and next token **w**:

$$p(w|h) = (c(w, h) + a) / (c(h) + a \cdot |V|)$$

h: the context is either the previous word (for the bigram model), or a bigram (for the trigram model).

w: the token(word) that comes after the context.

To train the models, we provide:

- **The vocabulary**, containing all valid words and special tokens.
- **Bigram counts** (**bigram_counts**), representing frequencies of all word pairs.
- **Unigram context counts** (**unigram_contexts**), used as context frequencies for bigram probability estimation.
- **Trigram counts** (**trigram_counts**), representing frequencies of all word triplets.
- **Bigram context counts** (**bigram_contexts**), used as context frequencies for trigram probability estimation.

These datasets are passed to the **train_ngram()** method of each model, ensuring that both bigram and trigram models share the same vocabulary and consistent frequency statistics.

Then, we print the most common of each model, using the **get_most_common** method, and we see the results:

```
Top 10 bigrams:
[('*UNK*', '*UNK*'), 1447),
 ('.', '*end*'), 634),
 ('the', '*UNK*'), 502),
 ('*UNK*', ','), 461),
 ('*UNK*', '.'), 418),
 ('*start*', '*UNK*'), 273),
 ('', '*UNK*'), 251),
 ('*UNK*', 'of'), 218),
 ('*UNK*', 'to'), 181),
 ('a', '*UNK*'), 162)]
```

```
Top 10 trigrams:
[('*UNK*', '.', '*end*'), 418),
 ('*UNK*', '*UNK*', '*UNK*'), 398),
 ('*start1*', '*start2*', '*UNK*'), 273),
 ('the', '*UNK*', '*UNK*'), 224),
 ('*UNK*', ', ', '*UNK*'), 189),
 ('*UNK*', '*UNK*', '.'), 166),
 ('*start1*', '*start2*', ''), 160),
 ('*UNK*', '*UNK*', ', '), 150),
 ('*UNK*', 'the', '*UNK*'), 145),
 ('*UNK*', 'to', '*UNK*'), 110)]
```

The presence of `*UNK*` tokens in the most common ngrams is reasonable, as these combinations of words are more likely to appear.

Fine-Tuning the Smoothing Parameter (α)

To prevent under- or over-smoothing, we fine-tune our alpha parameter via the `fine_tune` method in `NgramLanguageModel`. Smaller α values provide **less smoothing**, giving higher weight to observed ngrams, whereas larger α values provide **more smoothing**.

Our final results are as follows:

- **Bigrams:** Optimal Alpha: 0.0536, Best Perplexity: 13.6358
- **Trigrams:** Optimal Alpha: 0.0426, Best Perplexity: 18.8844.

Probability and Log-Probability Calculation

We now evaluate how well the trained language models estimate the likelihood of specific word sequences within the processed development corpus. Using the methods `prob` and `log_prob` from the `NgramLanguageModel` class we can compute the **probability** and **log₂ probability** for individual bigram and trigram instances respectively. This testing is done after we have fine-tuned our alpha.

```
P(*UNK*|The) = 0.812372 (α=0.054)
P(*UNK*|The,*UNK*) = 0.425668 (α=0.043)
```

It's apparent that the probability of one or two unknown words appearing after 'The' is pretty high. The trigram probability is lower as the context becomes more and more specific, and the probability mass is spread over more possible continuations.

Cross Entropy and Perplexity Estimation

The implemented function `compute_cross_entropy_perplexity` serves to evaluate the performance of the two trained language models by calculating their **cross-entropy (H)** and **perplexity (PPL)**. Any start token is not included as they do not represent observable words in any natural text. On the other hand, end tokens are included, as the model needs to be able to predict when a sentence ends on a specific word.

We iterate through all sentences in the processed test corpus and sum the logarithmic probabilities of each predicted token. The **cross-entropy** of the model is the negative average of the log probabilities, and it measures the average uncertainty of the model when predicting each word in the test data. Subsequently, the **perplexity** is derived as two to the power of cross-entropy, and it can be interpreted as the effective number of equally likely choices the model faces when predicting a token. We also compare tuned models to the **default $\alpha = 1.0$** baseline, reporting the relative improvement in perplexity.

The results are the following:

```
Bigram Model ( $\alpha=0.054$ ):  
Cross-Entropy: 3.689  
Perplexity: 12.894
```

```
Trigram Model ( $\alpha=0.043$ ):  
Cross-Entropy: 4.221  
Perplexity: 18.643
```

```
Comparison with default  $\alpha=1.0$ :  
Bigram: 0.77x improvement  
Trigram: 0.63x improvement
```

At first, it might seem counterintuitive for the trigram model- that normally performs better due to having longer context- to have higher cross-entropy and perplexity than the bigram one. However, we must consider that we used add-a smoothing which is not the most effective way to train a ngram model, but a rather simplistic one, and the presence of ***UNK*** tokens which reduces the diversity of the vocabulary. There are less informative contexts for trigrams, making possible predictions more uncertain.

To qualitatively assess the performance of both decoding strategies, we tested our **bigram** and **trigram** language models on several incomplete sentence prefixes from the **Brown (science_fiction)** corpus.

The examples below demonstrate how each model continues a given prefix using **greedy decoding** and **beam search** (*beam width* = 3). We show some interesting behavior cases.

Greedy decoding

Example 1: 'He has the...'

```
Prefix: He has the  
Bigram: He has the first time . ( $\alpha=0.054$ )  
Trigram: He has the first time . ( $\alpha=0.043$ )
```

Example 2: 'I was in...'

```
Prefix: I was in  
Bigram: I was in the first time . ( $\alpha=0.054$ )  
Trigram: I was in there . ( $\alpha=0.043$ )
```

Example 3: 'She was not...'

```
Prefix: She was not  
Bigram: She was not be the first time . ( $\alpha=0.054$ )  
Trigram: She was not , the one he thought with and the one he thought with and the one he thought with and the ( $\alpha=0.043$ )
```

Beam Search

Example 1: 'He has the...'

```
Prefix: He has the  
Bigram: He has the ship . ( $\alpha=0.054$ )  
Trigram: He has the first time . ( $\alpha=0.043$ )
```

Example 2: 'I was in...'

```
Prefix: I was in  
Bigram: I was in his mind he had been on the other '' . ( $\alpha=0.054$ )  
Trigram: I was in there . ( $\alpha=0.043$ )
```

Example 3: 'She was not...'

```
Prefix: She was not  
Bigram: She was not have been on the other '' . ( $\alpha=0.054$ )  
Trigram: She was not , for all the other '' . ( $\alpha=0.043$ )
```

- **Greedy decoding** produces short and repetitive completions since it only chooses the locally most probable next token.

- **Beam search**, by maintaining multiple hypotheses, often produces longer, more contextually diverse continuations (e.g., “*He has the ship*” vs. “*He has the first time*”).
- **Trigram models** generate more fluent text overall, but sometimes suffer from *looping* or *over-generation* (“the one he thought with and the one he thought with ...”), a known issue in higher-order n-gram models with simple smoothing.
- The presence of phrases like “*the first time*” or “*the ship*” reflects frequent patterns in the **Brown science_fiction** domain.
- The comparison **confirms** that **increasing context** ($n=3$) and using **beam search** both enhance **coherence**, although excessive repetition indicates the **limitations of Laplace smoothing**.